TOOLKIT · BACKEND ENGINEERING HANDBOOK

KUBERNETES COMPLETE HANDBOOK

Cluster Architecture, Pod Lifecycle, Networking & Services, Ingress, Storage, HPA, Security, Production Reliability & Incident Mastery

Phase	IV — Container Orchestration & Cloud Infrastructure
Tool ID	12 of 13
Curriculum Scope	Complete 3-Chunk Comprehensive Curriculum
Deliverable Type	Consolidated Tool Engineering Handbook (Release Build)
Source Authority	content/kubernetes/ (01, 02, and 03 markdown sources)
Target Audience	Backend Software Engineers, SDE-1 / SDE-2, Cloud Engineers, SREs
Engineering Focus	Control Plane & Kubelet, Declarative Reconciliation, Services & DNS, Ingress, PVCs/PVs, HPA & Resource Pressure, RBAC, Probes, Rollouts & Incident Triage

This authoritative engineering handbook provides a comprehensive, production-grade guide to **Kubernetes** for backend developers and cloud practitioners. Structured across three rigorous pedagogical layers—Mental Models, Practical Workflows, and Production Incident Mastery—it bridges the gap between running containers locally and operating resilient multi-service workloads in production. Readers master control-plane components (API server, etcd, scheduler, controller-manager) and worker architecture (kubelet, kube-proxy, CRI), declarative state reconciliation, Pod lifecycle and probes, ClusterIP/NodePort/LoadBalancer routing and CoreDNS discovery, Ingress controllers, ConfigMaps, Secrets, persistent volume claims, Horizontal Pod Autoscaling (HPA), PodDisruptionBudgets, rolling updates and rollbacks, least-privilege RBAC, NetworkPolicies, container security contexts, resource requests/limits, OOMKill diagnosis, and systematic incident response.

Curriculum Structure & Core Topics

Part I — Containers at Scale, Cluster Architecture & Core Mental Models: The orchestration problem, control plane vs worker nodes, etcd, scheduler, controller-manager, kubelet, container runtime, Pod abstractions, Deployments, ReplicaSets, declarative reconciliation loops, Pod lifecycle states, debugging Pod startup failures (CrashLoopBackOff, ImagePullBackOff), and local deployment workflows.

Part II — Services, Networking, Ingress, Configuration, Secrets & Persistent Storage: Pod IP ephemeral nature, Service types (ClusterIP, NodePort, LoadBalancer), EndpointSlices, CoreDNS service discovery, Ingress resources vs controllers, ConfigMaps, Secrets (Base64 vs encryption), storage architecture (PVs, PVCs, StorageClasses, access modes), and multi-service backend labs.

Part III — Production Reliability, Security, Autoscaling, Observability & Incident Mastery: Resource requests and limits, CPU throttling vs memory OOMKill, probe design (startup, readiness, liveness), Horizontal Pod Autoscaling (HPA), PodDisruptionBudgets, zero-downtime rolling updates and rollbacks, backward-compatible database migrations, RBAC (Roles, ClusterRoles, RoleBindings), ServiceAccounts, NetworkPolicies, container security contexts, graceful termination, metrics/logs/traces observability, and real-world production incident triage runbooks.

Part I: Containers at Scale, Cluster Architecture & Core Mental Models

Chunk 1 of 3 · Backend Engineering Handbook

Kubernetes — Chunk 1 of 3: Containers at Scale, Cluster Architecture & Core Mental Models

Backend Engineering Tools & Infrastructure Handbook

Source file:
`content/kubernetes/01-kubernetes-containers-cluster-architecture-core-mental-models.md`

Learning Objectives

By the end of this chapter, you should be able to:

- Explain why Kubernetes exists and what problems it solves.
- Understand cluster architecture and the responsibilities of control-plane and worker-node components.
- Distinguish Pods, containers, Deployments, ReplicaSets, Services, and Namespaces.
- Explain how Kubernetes schedules and runs a backend application.
- Understand declarative configuration and reconciliation.
- Deploy a Spring Boot API to a local Kubernetes cluster.
- Diagnose common Pod scheduling, startup, networking, and configuration failures.
- Answer Kubernetes fundamentals questions in backend engineering interviews.

This chapter establishes the mental models and first practical workflow. Chunks 2 and 3 will build on it with networking, storage, configuration, security, production reliability, scaling, troubleshooting, and interview mastery.

Layer 1 — Fundamentals & Mental Models

1. Why Kubernetes Exists

Imagine you have three backend services:

- Orders API — Spring Boot
- Payments API — Spring Boot
- Notifications worker — Java

Each runs in a container. Initially, you run them manually with Docker.

That works—until production grows.

You now need to answer:

- What happens if a container crashes?
- How do you run five copies of an API?
- How do you replace an unhealthy instance?
- How do you deploy a new version without taking down the service?
- How do containers discover one another?
- How do you allocate CPU and memory?
- How do you manage configuration across development, staging, and production?

Docker provides containerization. Kubernetes orchestrates containerized workloads across a cluster of machines.

Mental model: Docker packages and runs containers; Kubernetes manages the desired state and lifecycle of workloads across a cluster.

Kubernetes does not eliminate operational complexity. It gives you mechanisms for managing complexity consistently, but those mechanisms still require correct configuration, monitoring, security, and capacity planning.

2. What Is a Kubernetes Cluster?

A Kubernetes cluster consists of a **control plane** and one or more **worker nodes**.

Control-Plane Components

The control plane makes global cluster decisions and maintains desired state:

Component	Responsibility
API Server (`kube-apiserver`)	Entry point and primary interface for all Kubernetes API operations. Validates and configures data for Pods, Services, and Deployments.
etcd	Consistent, distributed, and highly-available key-value store holding the entire cluster state.
Scheduler (`kube-scheduler`)	Selects suitable worker nodes for newly created, unscheduled Pods based on resource constraints, affinity, and availability.
Controller Manager (`kube-controller-manager`)	Runs controller loops that continuously reconcile actual cluster state toward declared desired state.

Worker-Node Components

Worker nodes host and run the application container workloads:

Component	Responsibility
kubelet	The primary node agent that registers the node, communicates with the control plane, and instructs the container runtime to run assigned Pods.
Container runtime	Executes containers on the node, commonly through containerd via the Container Runtime Interface (CRI).
kube-proxy (or equivalent)	Implements Service traffic routing, maintaining network rules on nodes (via iptables, IPVS, or eBPF).
Pods	The smallest deployable units containing the application containers and shared volumes.

The exact networking implementation varies by cluster. Some environments use `kube-proxy`; others use a network implementation that replaces some or all of its functionality.

Control-plane components detailed

API server (`kube-apiserver`)

The primary interface for Kubernetes API operations. Tools such as `kubectl` communicate with the API server.

etcd

A consistent key-value store that holds Kubernetes cluster state. It is not your application's default database.

Scheduler (`kube-scheduler`)

Selects a suitable node for a Pod that has not yet been assigned to a node. It considers factors such as resource requests, scheduling constraints, and node availability.

Controller manager

Runs controllers that observe cluster state and take action to move the system toward its desired state.

Worker-node components detailed

kubelet

The node agent that communicates with the control plane and works with the container runtime to run assigned Pods.

Container runtime

Executes containers. Kubernetes commonly uses containerd through the Container Runtime Interface (CRI).

Networking components

Implement Pod networking and Service traffic routing. Their exact design depends on the cluster's networking solution.

3. The Declarative Model

One of Kubernetes' most important ideas is declarative configuration.

Instead of manually starting three containers and repeatedly checking whether they are alive, you declare what you want:

> Run three replicas of my Orders API using this image.

Kubernetes controllers continuously compare the declared state with the observed state and take corrective action where possible.

```
You declare desired state
          |
          v
    Kubernetes API
          |
          v
 Controllers observe state
          |
          v
 Create / replace / reconcile resources
          |
          v
 Actual state moves toward desired state
```

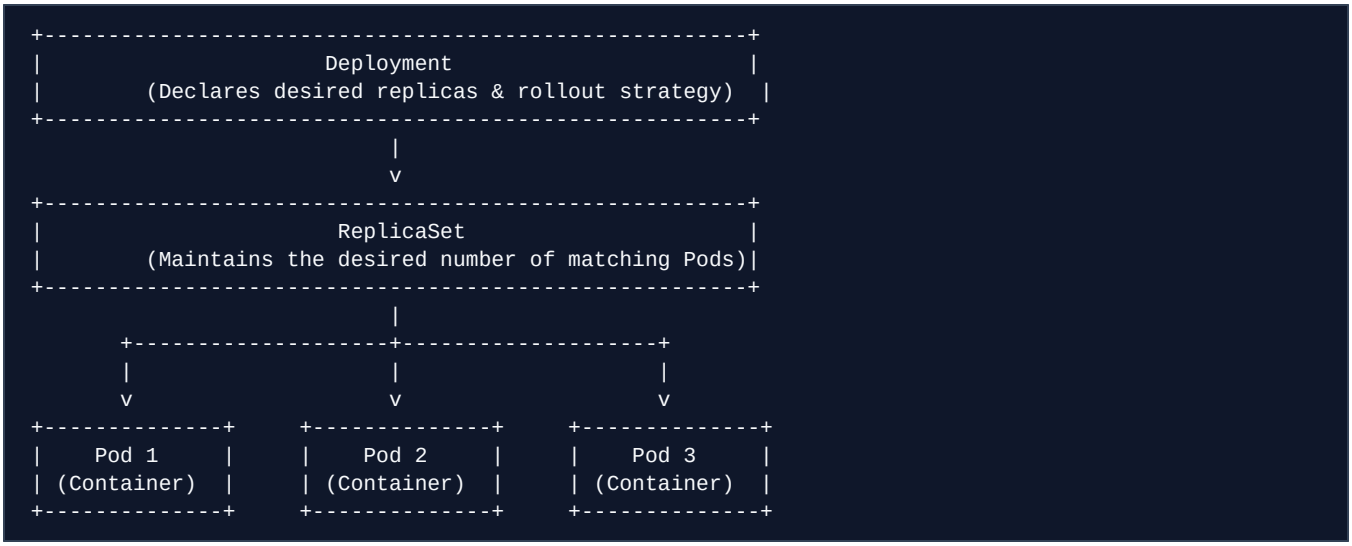
Suppose you request three replicas, but only two are running.

A Deployment's controller chain works to restore the desired replica count, subject to scheduling, resource availability, image availability, and other constraints.

Important distinction: Kubernetes is not magic self-healing. It can replace a failed Pod, but it cannot guarantee that the application inside the replacement will work correctly.

4. The Kubernetes Object Hierarchy

Several objects work together to run an application.



Pod

A Pod is Kubernetes' smallest deployable workload unit.

A Pod contains one or more containers that share the Pod's network namespace and can share mounted volumes.

Most ordinary backend workloads use one primary application container per Pod. Multiple containers are useful when they are tightly coupled, such as an application and a supporting sidecar.

A Pod is not simply a permanent container. Pods are replaceable, and a replacement Pod can have a different identity and IP address.

ReplicaSet

A ReplicaSet maintains the requested number of matching Pod replicas.

You will usually create a Deployment rather than manage a ReplicaSet directly.

Deployment

A Deployment manages stateless application rollout and rollback behavior through ReplicaSets.

It provides a declarative way to update application versions and maintain a requested replica count.

Service

Pods can be replaced, so their individual IP addresses are not a reliable long-term destination for clients.

A Service provides a stable network identity for a set of matching Pods.

We will explore Service networking and external access in Chunk 2.

Namespace

A Namespace groups namespaced Kubernetes resources inside a cluster.

Namespaces help organize environments and teams, but they are not a complete security boundary by themselves.

5. Container Versus Pod

Concept	Responsibility
Container	Runs an application process in an isolated environment
Pod	Provides the Kubernetes execution unit around one or more containers
ReplicaSet	Maintains the requested number of matching Pods
Deployment	Manages declarative updates and ReplicaSets
Service	Provides stable access to matching Pods
Namespace	Organizes namespaced resources

For example, if you deploy a Spring Boot API with three replicas, you usually have three Pods, each containing one application container.

You do not normally create one giant Pod with three copies of the same application inside it. The replicas should generally be separate Pods so Kubernetes can schedule and replace them independently.

6. Kubernetes Scheduling: What Happens After You Apply YAML?

Consider:

```
kubectl apply -f deployment.yaml
```

A simplified execution sequence is:

1. `kubectl` sends the desired object to the API server.
2. The API server validates and stores the accepted object.
3. The Deployment controller creates or updates a ReplicaSet.
4. The ReplicaSet controller creates the required Pods.
5. The scheduler selects nodes for unscheduled Pods.
6. The kubelet on each selected node observes its assigned Pods.
7. The container runtime pulls images when needed and starts containers.
8. The application initializes.
9. Readiness checks, if configured, determine whether the Pod is ready to serve traffic.

These steps are coordinated by asynchronous control loops. They do not happen as one synchronous transaction.

A successful `kubectl apply` means the configuration was accepted; it does not prove the application is healthy or reachable.

7. Desired State Versus Observed State

Suppose a Deployment specifies:

```
spec:
  replicas: 3
```

This means the desired number of replicas is three.

You might observe:

```
NAME                          READY   STATUS
orders-api-abc123-1           1/1     Running
orders-api-abc123-2           1/1     Running
orders-api-abc123-3           0/1     Running
```

Three Pods exist, but only two are ready.

That distinction matters:

- **Desired replicas:** how many replicas the configuration requests.
- **Existing replicas:** how many corresponding Pods exist.
- **Ready replicas:** how many satisfy readiness conditions.
- **Available replicas:** how many meet the Deployment's availability criteria.

A Pod can be running without being ready to serve traffic.

Interview insight: `Running` is a Pod phase, not a complete health assessment.

8. Kubernetes Is Not a Database or a Build Tool

Kubernetes is responsible for orchestration, not for replacing every other component in your stack.

Tool	Main responsibility
Maven	Builds and packages the Java application
Docker	Builds container images and provides container tooling

Tool	Main responsibility
Kubernetes	Orchestrates workloads and cluster resources
PostgreSQL	Stores relational application data
Redis	Provides in-memory data structures and caching capabilities
Kafka	Provides distributed event streaming
Prometheus	Collects and queries metrics
Grafana	Visualizes operational data

Kubernetes can run PostgreSQL, Redis, and Kafka, but running a service in Kubernetes does not automatically make that service highly available, backed up, or correctly configured.

Stateful workloads need additional architectural decisions around storage, recovery, replication, and operational ownership.

Layer 2 — Practical Engineering & Workflows

9. Set Up a Local Kubernetes Environment

For learning, use a local cluster rather than paying for a managed cloud cluster.

Common options include:

- **Kind:** runs Kubernetes nodes in Docker containers.
- **Minikube:** creates a local Kubernetes environment.
- **Docker Desktop Kubernetes:** available in supported Docker Desktop configurations.

Choose one environment and keep its setup consistent throughout the handbook.

Example: Kind

Check prerequisites:

```
docker --version
kubectl version --client
kind version
```

Create a cluster:

```
kind create cluster --name backend-lab
kubectl cluster-info
kubectl get nodes
```

Inspect the current context:

```
kubectl config current-context
kubectl config get-contexts
```

A Kubernetes context identifies a cluster, a user or credentials, and optionally a default Namespace.

Safety rule: Before applying or deleting resources, confirm which context is active. The same command can affect a local cluster or a production cluster depending on the context.

10. Build a Minimal Spring Boot Image

Kubernetes runs container images; it does not compile your Java source code into a production application automatically.

From your Spring Boot project:

```
./mvnw clean package
```

Assuming the build produces a JAR under `target/`, a simple development Dockerfile is:

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Build the image:

```
docker build -t orders-api:1.0 .
```

This is a minimal teaching example, not a hardened production Dockerfile. Production images should use controlled base-image versions, appropriate non-root execution, and a deliberate patching and scanning process.

Make the image available to Kind

Kind nodes have their own image caches. A locally built image is not automatically available inside the cluster.

Load it:

```
kind load docker-image orders-api:1.0 --name backend-lab
```

This prevents Kubernetes from trying to fetch the local development image from a remote registry, provided the Pod image settings are configured appropriately.

11. Write a Kubernetes Deployment

Create `k8s/deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: orders-api
  labels:
    app: orders-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: orders-api
  template:
    metadata:
      labels:
        app: orders-api
    spec:
      containers:
        - name: orders-api
          image: orders-api:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8080
          resources:
            requests:
              cpu: "100m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
```

Understand the important fields:

- **`apiVersion`**: API group and version for the resource.
- **`kind`**: resource type.
- **`metadata`**: name and labels.
- **`replicas`**: desired Pod count.
- **`selector`**: determines which Pods the Deployment manages.
- **`template`**: defines how new Pods should be created.
- **`image`**: application container image.
- **`containerPort`**: documents the port the container is expected to use.
- **`resources.requests`**: resource quantities used for scheduling and related decisions.
- **`resources.limits`**: resource constraints applied by the node's container runtime and operating system mechanisms.

The Deployment selector must match the labels on its Pod template.

Apply the Deployment

```
kubectl apply -f k8s/deployment.yaml
kubectl get deployments
kubectl get replicasets
kubectl get pods -o wide
```

Watch rollout progress:

```
kubectl rollout status deployment/orders-api
```

Inspect the Deployment:

```
kubectl describe deployment orders-api
```

If it does not become ready, inspect the Pods and their events rather than repeatedly reapplying the same manifest.

12. Expose the Application Locally

Create `k8s/service.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: orders-api
spec:
  type: ClusterIP
  selector:
    app: orders-api
  ports:
    - name: http
      port: 80
      targetPort: 8080
```

Apply it:

```
kubectl apply -f k8s/service.yaml
kubectl get service orders-api
```

`ClusterIP` creates a Service reachable from within the cluster. It does not automatically expose the API to your host machine.

For a local test, use port forwarding:

```
kubectl port-forward service/orders-api 8080:80
```

Then, in another terminal:

```
curl -i http://localhost:8080/
```

Replace `/` with an endpoint that actually exists in your Spring Boot application. A `404` from an undefined route does not necessarily indicate a Kubernetes networking failure.

Port forwarding is a convenient development technique, not a general production exposure strategy.

13. The Most Useful Initial Commands

The essential command field guide for cluster inspection and workload debugging:

Category	Primary Commands	Operational Purpose
Cluster	`kubectl cluster-info` `kubectl get nodes`	Check cluster access, control-plane reachability, and node availability.

Workloads	`kubectl get deployments`
	`kubectl get replicasets`
	`kubectl get pods -o wide`
		Compare desired replicas with actual running Pods and inspect their IP and node placement.

Details	`kubectl describe pod <pod-name>`	Inspect detailed conditions, scheduling decisions, probes, lifecycle events, and container exit codes.
Logs	`kubectl logs <pod-name>` `kubectl logs <pod-name> --previous`	Stream current application output or examine logs from a previously crashed container instance.
Events	`kubectl get events --sort-by=.metadata.creationTimestamp`	Review cluster-wide scheduling, scaling, and failure events in chronological order.
Rollout	`kubectl rollout status deployment/orders-api` `kubectl rollout history deployment/orders-api`	Monitor rollout transitions and inspect previous revision histories.

14. Common Failure Modes

`ImagePullBackOff`

Meaning: Kubernetes cannot successfully obtain the requested image.

Check:

```
kubectl describe pod <pod-name>
```

Look for events indicating:

- Image not found.
- Registry authentication failure.
- Network or DNS failure.
- Incorrect tag.
- Local image not loaded into the cluster.

For Kind, verify that the local image was loaded into the correct cluster.

`CrashLoopBackOff`

Meaning: The container repeatedly terminates and Kubernetes backs off before restarting it.

Investigate:

```
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
kubectl describe pod <pod-name>
```

Possible causes include invalid configuration, missing environment variables, failed startup dependencies, application exceptions, or resource constraints.

The status describes a restart pattern, not the root cause.

`Pending`

A Pod may remain pending because:

- No node has sufficient requested resources.
- Scheduling constraints cannot be satisfied.
- A required volume cannot be attached or provisioned.
- A scheduling policy prevents placement.

Inspect the Pod's events and scheduling conditions.

`Running` but not `Ready`

The container may be running while the application is still initializing, failing readiness checks, or unable to serve traffic correctly.

Check Pod conditions, container logs, probe configuration, and application health.

Service has no working endpoints

Check that the Service selector matches the Pod labels:

```
kubectl get service orders-api -o yaml
kubectl get pods --show-labels
kubectl get endpointslice
```

A Service can exist even when no ready Pods match its selector.

Layer 3 — Production Engineering & Interview Mastery

15. Resource Requests and Limits

Kubernetes resource configuration influences both scheduling and runtime behavior.

Requests

A resource request describes the amount of a resource the scheduler should account for when placing the Pod.

For example:

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
```

`250m` CPU means one quarter of a CPU core.

Limits

A resource limit constrains resource usage.

```
resources:
  limits:
    cpu: "1"
    memory: "512Mi"
```

CPU limits can cause throttling when the container attempts to exceed its permitted CPU usage. Memory-limit violations can result in an out-of-memory termination.

A Pod that requests too much memory may remain unschedulable. A Pod with a memory limit that is too low may repeatedly restart under load.

Resource values should be based on observed application behavior, workload testing, and capacity planning rather than arbitrary numbers.

16. Why Multiple Replicas Matter

Three replicas can improve availability and throughput, but only if the surrounding architecture supports them.

Ask:

- Can requests reach all replicas?
- Can replicas run on different nodes?
- Does the application rely on local in-memory state?
- Can the database handle concurrent connections from every replica?
- Do all replicas use compatible configuration?
- Can a rolling deployment maintain enough available replicas?
- Does the service remain healthy if one node fails?

Important: Three replicas on the same worker node do not provide the same node-failure protection as replicas distributed across independent nodes.

Replication of application Pods also does not automatically replicate database state or make a stateful dependency highly available.

17. What Kubernetes Does Not Guarantee

Kubernetes can help maintain declared workload state, but it does not automatically guarantee:

- Zero downtime for every deployment.
- Correct application logic.
- Database consistency.
- Durable storage.
- Safe secret management.
- Cross-node fault tolerance.
- Sufficient cluster capacity.
- Correct health checks.
- Secure network access.

- Reliable backups and recovery.

These outcomes depend on the application's design, Kubernetes configuration, cluster infrastructure, and operational practices.

18. Production Incident: The Deployment Never Becomes Ready

Imagine that you deploy version `1.1` of your Orders API.

The Deployment exists, but the rollout stalls.

Use a disciplined workflow:

1. Check rollout status.
2. Inspect Deployment and ReplicaSet events.
3. Inspect the Pods created by the new ReplicaSet.
4. Check container logs.
5. Check the image reference and image availability.
6. Review environment variables and external dependencies.
7. Review resource requests, limits, and scheduling constraints.
8. Check readiness probes if configured.
9. Compare the new version with the previous healthy revision.
10. Roll back if the new version is causing an active production incident and rollback is safe.

Useful commands:

```
kubectl rollout status deployment/orders-api
kubectl get replicasets
kubectl get pods -o wide
kubectl describe deployment orders-api
kubectl describe pod <pod-name>
kubectl logs <pod-name> --previous
```

If you decide to roll back:

```
kubectl rollout undo deployment/orders-api
kubectl rollout status deployment/orders-api
```

Verify the application and its dependencies after the rollback. A successful rollout command does not prove that every user-facing operation is functioning correctly.

19. Kubernetes Interview Questions

1. What is Kubernetes?

A container-orchestration platform that manages workloads and cluster resources through declarative APIs and controllers.

2. What is a cluster?

A Kubernetes environment consisting of a control plane and worker nodes.

3. What does the API server do?

It exposes the Kubernetes API and serves as the main interface for cluster operations.

4. What is etcd?

The consistent key-value store used to persist Kubernetes cluster state.

5. What does the scheduler do?

It selects an appropriate node for Pods that have not yet been scheduled.

6. What is a Pod?

The smallest deployable Kubernetes workload unit, containing one or more closely related containers.

7. Why use a Deployment instead of creating Pods manually?

A Deployment provides declarative replica management, rollout behavior, and revision history through ReplicaSets.

8. What is a ReplicaSet?

A controller-managed resource that maintains the desired number of matching Pods.

9. Why do Services exist?

They provide a stable network identity for accessing a changing set of matching Pods.

10. What is a Namespace?

A mechanism for organizing namespaced resources within a cluster. It is not, by itself, a complete security boundary.

11. What is declarative configuration?

Describing the desired state and allowing Kubernetes controllers to reconcile observed state toward it.

12. Does `kubectl apply` mean the application is healthy?

No. It means the configuration was accepted or updated. Workloads may still be unscheduled, starting, crashing, or not ready.

13. What is the difference between `Running` and `Ready`?

`Running` describes the Pod phase. `Ready` indicates whether the Pod currently satisfies the conditions for being considered ready to serve traffic.

14. What is `CrashLoopBackOff`?

A restart-backoff condition seen when a container repeatedly terminates and Kubernetes delays subsequent restart attempts.

15. What causes `ImagePullBackOff`?

Failures obtaining an image, such as an invalid image reference, unavailable registry, authentication problem, or missing local image.

16. What is the purpose of resource requests?

They inform scheduling decisions and describe the resources the workload requests.

17. What can happen when a container exceeds its memory limit?

It may be terminated due to an out-of-memory condition.

18. Does Kubernetes automatically make an application highly available?

No. Kubernetes provides orchestration mechanisms, but availability also depends on replica placement, dependencies, health checks, capacity, and application design.

19. Why is a local image sometimes unavailable to a local Kubernetes cluster?

The cluster's nodes may have separate image caches from the host's Docker daemon. The image must be loaded into the cluster or made available from a registry.

20. How do you investigate a failed rollout?

Inspect rollout status, ReplicaSets, Pod events, image availability, application logs, configuration, resources, and readiness behavior. Roll back if needed and safe.

20. Hands-On Capstone

Deploy a Spring Boot Orders API to your local Kubernetes cluster.

Tasks

- [ ] Create a local cluster.
- [ ] Build the application JAR.
- [ ] Build a container image.
- [ ] Load the image into the local cluster or publish it to a registry.
- [ ] Create a Deployment with three replicas.
- [ ] Configure resource requests and limits.
- [ ] Create a ClusterIP Service.
- [ ] Verify that the Pods become ready.
- [ ] Use port forwarding to test a real API endpoint.
- [ ] Intentionally use an incorrect image tag and diagnose the failure.
- [ ] Inspect logs for a container that exits.
- [ ] Explain the difference between desired, existing, ready, and available replicas.
- [ ] Roll back a test Deployment revision.
- [ ] Document the troubleshooting sequence and evidence.

Mastery standard

You should be able to explain how a declarative Deployment becomes running Pods, trace the responsibilities of the control plane and worker nodes, and diagnose the first layer of Kubernetes workload failures without guessing.

Final Mastery Checklist

- [ ] Explain Kubernetes' role relative to Docker.
- [ ] Describe the control plane and worker-node components.
- [ ] Explain the API server, etcd, scheduler, controllers, and kubelet.
- [ ] Distinguish Pods, ReplicaSets, Deployments, Services, and Namespaces.
- [ ] Explain desired state and reconciliation.
- [ ] Deploy a Spring Boot API locally.
- [ ] Explain requests versus limits.

- [ ] Diagnose `Pending`, `CrashLoopBackOff`, and `ImagePullBackOff`.
- [ ] Understand why `Running` does not necessarily mean `Ready`.
- [ ] Explain why replicas alone do not guarantee high availability.
- [ ] Perform a controlled rollback.
- [ ] Complete the hands-on capstone.

Kubernetes Handbook Progress

This is **Chunk 1 of 3**. The remaining planned chunks are:

- **Chunk 2:** Services, Networking, Ingress, Configuration, Secrets & Persistent Storage.
- **Chunk 3:** Production Reliability, Security, Autoscaling, Observability, Incident Mastery & Interview Preparation.

Save this chapter to:

`content/kubernetes/01-kubernetes-containers-cluster-architecture-core-mental-models.md`

Do not generate the Kubernetes PDF yet. Save and verify all three planned chapters first, then generate the single complete handbook at:

`docs/kubernetes/BEH_KUBERNETES_COMPLETE_HANDBOOK.pdf`

After all three source chapters and the final PDF are saved and verified, commit using the exact message:

`Kubernetes done`

Do not claim repository changes, PDF generation, or Git commits until they have actually been performed and verified.

Part II: Services, Networking, Ingress, Configuration, Secrets & Persistent Storage

Chunk 2 of 3 · Backend Engineering Handbook

Kubernetes — Chunk 2 of 3: Services, Networking, Ingress, Configuration, Secrets & Persistent Storage

Backend Engineering Tools & Infrastructure Handbook

Source file: `content/kubernetes/02-kubernetes-services-networking-ingress-configuration-storage.md`

Learning Objectives

By the end of this chapter, you should be able to:

- Explain how Kubernetes networking connects Pods, Services, and external clients.
- Distinguish ClusterIP, NodePort, and LoadBalancer Services.
- Understand DNS-based service discovery.
- Explain Ingress and the role of an Ingress controller.
- Manage application configuration with ConfigMaps and Secrets.
- Understand PersistentVolumes, PersistentVolumeClaims, and StorageClasses.
- Deploy a multi-component backend application locally.
- Troubleshoot connectivity, configuration, and storage failures.
- Answer common Kubernetes networking and configuration interview questions.

This chapter covers all three required layers:

1. **Fundamentals & Mental Models**
2. **Practical Engineering & Workflows**
3. **Production Engineering & Interview Mastery**

Layer 1 — Fundamentals & Mental Models

1. The Kubernetes Networking Problem

In a backend application, several components need to communicate:

- A client sends a request to the Orders API.
- The Orders API queries PostgreSQL.
- The API uses Redis for caching.

- An asynchronous worker processes events from Kafka.

Kubernetes Pods are replaceable. Their IP addresses can change when they are recreated. Applications therefore need a stable way to locate other components.

Kubernetes networking provides mechanisms for:

- Pod-to-Pod communication.
- Stable access to a group of Pods.
- Service discovery through DNS.
- External access to applications.
- Network isolation through supported networking policies.

Mental model: Pods are the changing destinations; Services provide stable access to a selected set of Pods.

2. How a Service Routes Traffic



A typical Service selects Pods using labels. Kubernetes maintains endpoint information for matching Pods, commonly through EndpointSlices.

The cluster's networking implementation handles traffic delivery. Depending on the cluster, this may involve `kube-proxy` or an alternative implementation.

A Service does not repair a broken application. It provides a stable access mechanism to eligible endpoints.

3. Kubernetes Service Types

Service Type	Routing Mechanism	Typical Production Use Case
ClusterIP *(Default)*	Exposes a stable virtual IP reachable only from within the cluster.	Internal backend communication (e.g. Orders API querying PostgreSQL or Redis).
NodePort	Exposes the Service on a static high port (`30000-32767`) across every cluster node's IP.	Direct node debugging, local development clusters, or non-cloud bare-metal setups.

Service Type	Routing Mechanism	Typical Production Use Case
LoadBalancer	Provisions an external cloud load balancer (e.g. AWS NLB/ALB, GCP Cloud LB) pointing to node endpoints.	Publicly exposing critical production entrypoints when Ingress is not used.

Example: Internal API Service

```
apiVersion: v1
kind: Service
metadata:
  name: orders-api
spec:
  type: ClusterIP
  selector:
    app: orders-api
  ports:
    - name: http
      port: 80
      targetPort: 8080
```

Here:

- `port` is the port exposed by the Service.
- `targetPort` is the destination port on the selected Pod.
- `selector` identifies Pods using their labels.

If the application listens on port 8080, the Service can accept traffic on port 80 and forward it to port 8080.

4. Kubernetes DNS and Service Discovery

Kubernetes clusters commonly provide DNS-based discovery for Services.

Within the same Namespace, a client can usually connect using the Service name:

```
http://orders-api
```

A fully qualified Service DNS name typically follows this pattern:

```
<service>.<namespace>.svc.cluster.local
```

For example:

```
http://orders-api.production.svc.cluster.local
```

The exact cluster domain can be configured, so `cluster.local` is common rather than universal.

Example: Backend connection configuration

Suppose the `orders-api` application needs Redis.

Inside the same Namespace, its Redis URL might be:

```
redis://redis:6379
```

If Redis is in another Namespace:

```
redis://redis.data.svc.cluster.local:6379
```

This works only if a matching Service exists, DNS is functioning, network policies permit the connection, and the destination is reachable.

Common mistake: Using `localhost` to connect to another Pod.

Inside a container, `localhost` refers to that container's own network namespace. It does not automatically mean another Pod, another Service, or your laptop.

5. Ingress: HTTP and HTTPS Entry Routing

A Service gives you a stable endpoint. Ingress provides a declarative way to describe HTTP and HTTPS routing from outside the cluster to Services.

For example:

```
api.example.com/orders   → orders-api Service
api.example.com/payments → payments-api Service
```

The architecture commonly looks like this:

```
External Client
      |
      v
Ingress Controller
      |
      +------> orders-api Service ------> Orders Pods
      |
      +------> payments-api Service ----> Payments Pods
```

Ingress versus Ingress controller

These are not the same thing.

- **Ingress resource:** Declares routing rules for HTTP and HTTPS traffic.
- **Ingress controller:** Implements those rules by configuring or operating the actual traffic-routing mechanism.

Creating an Ingress resource does not guarantee that traffic is routed. A suitable controller must be installed and configured, and the environment must provide the required network access.

Kubernetes also supports the Gateway API, which offers a richer model for traffic management. This handbook focuses on Ingress for foundational understanding.

6. ConfigMaps and Secrets

Applications need configuration that changes between environments.

Examples include:

- Database hostnames.
- Port numbers.
- Feature flags.
- External service URLs.
- Database credentials.
- API tokens.

Hardcoding every setting into an image makes deployments difficult to manage. Kubernetes provides ConfigMaps and Secrets to separate configuration from application images.

ConfigMap

A ConfigMap stores non-confidential configuration.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: orders-config
data:
  APP_ENV: "development"
  REDIS_HOST: "redis"
  REDIS_PORT: "6379"
```

A Deployment can inject these values into a container:

```
envFrom:
  - configMapRef:
      name: orders-config
```

Secret

A Secret is designed for confidential configuration, such as credentials or tokens.

```
apiVersion: v1
kind: Secret
metadata:
  name: orders-db-secret
type: Opaque
stringData:
  DB_USERNAME: app_user
  DB_PASSWORD: replace-with-a-development-only-password
```

Inject the values:

```
env:
  - name: DB_USERNAME
    valueFrom:
      secretKeyRef:
        name: orders-db-secret
        key: DB_USERNAME
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: orders-db-secret
        key: DB_PASSWORD
```

The example password is a placeholder. Do not use it as a real credential.

Security warning: Kubernetes Secrets are not automatically encrypted end-to-end merely because they are called Secrets. Their data may be stored in etcd, and access depends on cluster configuration and authorization. Use least-privilege RBAC, encryption at rest where supported, secure delivery workflows, and an appropriate secret-management solution.

Base64 encoding is not encryption.

7. Persistent Storage: Why Containers Need More Than a Filesystem

A container's writable filesystem is not an appropriate default for durable application data.

Consider a PostgreSQL Pod that stores database files only in its writable container layer. If the Pod is replaced, the database files may be lost.

Kubernetes storage abstractions separate the lifecycle of a workload from the lifecycle of its storage.

Resource	Purpose
PersistentVolume (PV)	Represents provisioned storage available to the cluster
PersistentVolumeClaim (PVC)	Requests storage with particular requirements
StorageClass	Defines a storage provisioning class and its behavior
Volume mount	Makes a volume accessible inside a container

A PVC can request storage without the application needing to know every infrastructure-specific detail.

The actual behavior depends on the cluster's storage provisioner and volume configuration.

Mental model

- A Pod is an application execution unit.
- A PVC is a storage request.
- A PV represents storage made available to satisfy a claim.
- A StorageClass describes how storage can be provisioned.

Important: Persistent storage is not the same as a backup. A persistent volume may survive Pod replacement but still be lost through deletion, corruption, infrastructure failure, or an incorrect storage operation.

Layer 2 — Practical Engineering & Workflows

8. Lab: Deploy a Multi-Service Backend

We will extend the local Kubernetes cluster from Chunk 1.

The learning architecture:

```
Local Client
    |
    | port-forward
    v
orders-api Service
    |
    v
Orders API Pods
    |
    +------> redis Service
```

This lab focuses on Service discovery, configuration, and connectivity. It does not attempt to create a production-grade Redis deployment.

Step 1 — Verify the cluster

```
kubectl config current-context
kubectl get nodes
kubectl get deployments
kubectl get pods -o wide
```

Confirm that the active context is your local learning cluster before continuing.

Step 2 — Deploy Redis for the lab

Create **`k8s/redis.yaml`**:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:7
          ports:
            - name: redis
              containerPort: 6379
```

Create its Service:

```
apiVersion: v1
kind: Service
metadata:
  name: redis
spec:
  type: ClusterIP
  selector:
    app: redis
  ports:
    - name: redis
      port: 6379
      targetPort: 6379
```

Save these resources in a single `redis.yaml` file separated by `---`, or use separate files.

Apply the resources:

```
kubectl apply -f k8s/redis.yaml
kubectl get deployment redis
kubectl get service redis
kubectl get pods -l app=redis
```

This Redis configuration is intentionally minimal. It has no durable storage, authentication, or production hardening. Do not use it for important data.

Step 3 — Configure the Orders API

Create `k8s/orders-config.yaml`:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: orders-config
data:
  APP_ENV: "development"
  REDIS_HOST: "redis"
  REDIS_PORT: "6379"
```

Apply it:

```
kubectl apply -f k8s/orders-config.yaml
```

Update the Orders API Deployment to consume it:

```
envFrom:
  - configMapRef:
      name: orders-config
```

Place `envFrom` under the application container definition, at the same level as `image`, `ports`, and `resources`.

Your Spring Boot application must actually read `REDIS_HOST` and `REDIS_PORT` for this configuration to affect its Redis connection.

Step 4 — Verify Service discovery

Inspect the Service and endpoints:

```
kubectl get service redis
kubectl get endpointslice
kubectl get pods -l app=redis --show-labels
```

If you have a suitable debugging container in the cluster, test DNS resolution and TCP connectivity from it.

For example, a temporary diagnostic Pod could use a networking utility image approved for your environment. Keep diagnostic images controlled and remove temporary resources afterward.

Inside a Pod in the same Namespace, DNS resolution should normally resolve `redis` to the Service's cluster address.

If resolution fails, inspect DNS configuration, Namespace, Service existence, and cluster DNS health.

Step 5 — Test the API

Use the Service from the local host through port forwarding:

```
kubectl port-forward service/orders-api 8080:80
```

In a separate terminal, test a real application route:

```
curl -i http://localhost:8080/actuator/health
```

This assumes Spring Boot Actuator is installed and the health endpoint is enabled and exposed. Otherwise, use an endpoint that exists in your application.

A successful health response does not necessarily prove Redis operations work. Test the API path that actually exercises Redis if Redis integration is part of the application.

9. Lab: Configure a Secret Safely

For a local exercise, you can create a Secret interactively rather than committing credentials to YAML.

```
kubectl create secret generic orders-db-secret \
  --from-literal=DB_USERNAME=app_user \
  --from-literal=DB_PASSWORD='local-only-test-password'
```

The example is for a disposable local environment only.

Reference the Secret in the Deployment using `secretKeyRef`, as shown earlier.

Verify the Secret exists:

```
kubectl get secret orders-db-secret
kubectl describe secret orders-db-secret
```

Avoid printing secret values to terminal logs or copying them into tickets.

For production, prefer an approved secret-management workflow and avoid passing real credentials through shell history or process arguments.

10. Lab: Add Persistent Storage

For a local cluster with a working dynamic storage provisioner, create a PVC:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Apply and inspect it:

```
kubectl apply -f k8s/app-data-pvc.yaml
kubectl get pvc
kubectl describe pvc app-data
```

The claim may remain **`Pending`** if the cluster lacks a suitable StorageClass or provisioner. That is a storage-provisioning problem, not necessarily an application problem.

To use the claim, mount it into a Pod:

```
volumeMounts:
  - name: app-data
    mountPath: /var/lib/app-data

volumes:
  - name: app-data
    persistentVolumeClaim:
      claimName: app-data
```

The **`volumeMounts`** field belongs inside the container specification; **`volumes`** belongs inside the Pod specification.

Use a path appropriate for your application. Do not mount an arbitrary volume over application files or database directories without understanding the implications.

11. Troubleshooting Service Connectivity

Use this sequence when one Pod cannot reach another application.

A. Confirm the destination Service exists

```
kubectl get services
kubectl describe service redis
```

B. Confirm the selector matches Pod labels

```
kubectl get pods --show-labels
kubectl get service redis -o yaml
```

A selector mismatch can result in no matching endpoints.

C. Inspect EndpointSlices

```
kubectl get endpointslice
```

Confirm that the Service has the expected endpoint addresses and ports.

D. Check DNS from the client Pod

Use a diagnostic container to resolve the Service name.

If DNS fails, check the name, Namespace, cluster DNS, and network configuration.

E. Check the application listener

Confirm the destination application is actually listening on the expected port. A Pod's `containerPort` declaration does not force an application to bind to that port.

F. Check network policies

If the cluster uses NetworkPolicies, confirm that the policies permit the required ingress and egress traffic and that the networking implementation enforces them.

G. Inspect logs and events

```
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl get events --sort-by=.metadata.creationTimestamp
```

Debugging principle: Follow the request path one boundary at a time: DNS, Service, endpoints, network path, listening socket, and application behavior.

12. Troubleshooting Configuration and Storage

ConfigMap changes do not appear in the application

Possible explanations:

- The Pod uses environment variables populated at startup.
- The Deployment has not recreated the Pod.
- The application reads a different variable name.
- The application caches configuration.
- The ConfigMap is mounted as a file and the application does not reload it.

Changing a ConfigMap does not guarantee that an application automatically reloads its settings. Confirm the configuration delivery method and restart or reload the application as required.

Secret reference fails

Check:

- Secret name and Namespace.
- Key name spelling.
- Pod specification.
- Service account permissions and relevant policy.
- Application variable names.
- Whether the secret exists before the Pod starts.

Do not dump secret values into logs as a debugging shortcut.

PVC remains pending

Check:

```
kubectl get pvc
kubectl describe pvc app-data
kubectl get storageclass
kubectl get events --sort-by=.metadata.creationTimestamp
```

Potential causes include missing StorageClasses, unsupported access modes, capacity constraints, and provisioner failures.

Pod starts but cannot write to its volume

Investigate volume mounts, filesystem permissions, security context, ownership, access mode, and the application's write path.

A bound PVC alone does not guarantee that the process has permission to write to the mounted directory.

Layer 3 — Production Engineering & Interview Mastery

13. Networking Reliability

Production networking requires more than creating a Service.

Consider:

- Is the Service selecting the intended Pods?
- Are only appropriate Pods considered ready endpoints?
- Does DNS resolve consistently?
- Are ports aligned across the application, container, and Service?
- Do network policies permit the expected traffic?
- Can the destination handle connection volume?
- Are connection timeouts and retry behavior appropriate?
- Is external traffic protected by the intended TLS and access-control design?

Readiness versus liveness

These probes have different jobs:

- **Readiness probe:** Determines whether the container should be considered ready to serve traffic.
- **Liveness probe:** Determines whether the container should be restarted because it is unhealthy in a way that warrants restart.
- **Startup probe:** Gives a slow-starting application time to initialize before liveness and readiness checks take effect in the usual configured pattern.

An overly aggressive liveness probe can repeatedly restart a slow but recoverable application. A readiness probe that is too permissive can route traffic to an application that cannot serve requests.

Probe design is an application-reliability decision, not merely YAML configuration.

14. Ingress and TLS Considerations

In production, HTTP routing and TLS termination need a deliberate design.

A typical request path may be:

```
Client
  |
  | HTTPS
  v
External Load Balancer
  |
  v
Ingress Controller
  |
  v
Service
  |
  v
Application Pod
```

The exact path depends on the environment. TLS can terminate at different points, and the security of traffic between components must be considered separately.

Operational requirements include:

- A supported Ingress controller.
- Correct host and path rules.
- Valid certificates and renewal procedures.
- Correct backend Service ports.
- Appropriate request-size and timeout policies.
- Safe external exposure.
- Observability for routing failures.

A `404` from the Ingress layer, a `502` from a proxy, and a connection timeout can indicate different failure points. Identify which component generated the response before changing configuration.

15. Secrets and Access Control in Production

Use least privilege.

- Give workloads only the permissions they need.
- Avoid giving application Pods broad Kubernetes API permissions.
- Restrict who can read Secrets.
- Enable and verify encryption at rest where supported.
- Rotate credentials and revoke compromised values.
- Avoid putting credentials into container images or Git.
- Prefer a managed secret store or approved secret operator where appropriate.
- Audit access to sensitive resources.

Kubernetes RBAC controls API permissions; it does not automatically restrict every possible route to sensitive data. For example, access to create a Pod in a Namespace may, depending on permissions and configuration, create indirect ways to access mounted Secrets.

Security reviews should consider the effective permissions and workload capabilities, not only whether a user has direct permission to read a Secret.

16. Stateful Workloads and Persistent Storage

For databases and other stateful systems, the following questions matter:

- What happens when a Pod is recreated?
- Does storage survive Pod replacement?
- Is the storage replicated?
- What is the backup strategy?
- Can the application recover after corruption?
- How does volume attachment behave during rescheduling?
- What are the storage performance characteristics?
- Is recovery tested?

A PVC provides a storage abstraction, not a complete data-protection strategy.

StatefulSets can help manage stable identities and storage associations for stateful workloads, but they do not automatically make a database cluster highly available or consistent.

For production databases, choose the operational model deliberately: a managed database, a well-operated stateful deployment, or another architecture appropriate to the team's capabilities.

17. Production Incident Simulations

Incident A — Orders API cannot connect to Redis

Symptoms: API requests fail when accessing cached data.

Investigate:

1. Confirm the Redis Service exists.
2. Verify Service selectors and EndpointSlices.
3. Confirm the Redis Pod is ready.

4. Resolve the Service DNS name from the API Pod.
5. Test TCP connectivity to the expected port.
6. Inspect NetworkPolicies.
7. Check the application connection URL and credentials.
8. Review Redis logs and API exception details.

Likely causes: Incorrect hostname, wrong Namespace, missing endpoints, incorrect port, network policy restrictions, authentication failure, or a Redis process that is not ready.

Lesson: A successful DNS lookup proves only name resolution, not that the destination accepts connections.

Incident B — Deployment is healthy, but external users receive 502 responses

Investigate:

1. Confirm the external load balancer and Ingress controller are healthy.
2. Inspect the Ingress resource and controller events.
3. Check the backend Service name and port.
4. Inspect EndpointSlices.
5. Verify application listening ports.
6. Check readiness and startup behavior.
7. Inspect proxy logs and application logs.
8. Test connectivity at each layer.

Likely causes: Wrong backend port, missing endpoints, unhealthy Pods, proxy timeouts, or incorrect routing configuration.

Lesson: Start at the layer where the failure appears and trace the request inward.

Incident C — Application configuration does not change after a ConfigMap update

Investigate:

1. Verify the ConfigMap's current content.
2. Check whether values are injected as environment variables or mounted files.
3. Determine whether the Pod must be recreated.
4. Confirm the application uses the expected variable or file.
5. Check whether the application caches configuration.
6. Verify the running Pod's effective configuration without exposing secrets.

Likely cause: Environment variables are populated when the container starts, so existing processes may continue using their old values.

Lesson: Configuration storage and configuration activation are different operations.

Incident D — Database Pod is recreated and data disappears

Investigate:

1. Determine where the database wrote its data.
2. Inspect the Pod's volume mounts.
3. Check whether a PVC was used.

4. Inspect the PVC and PV lifecycle.
5. Check whether the database data directory was mounted correctly.
6. Determine whether data was deleted or merely became inaccessible.
7. Inspect backup and restore procedures.

Likely cause: Data was stored only in ephemeral container storage, or the persistent volume was incorrectly configured.

Lesson: Storage persistence, replication, and backup are separate requirements.

18. Twenty Interview Questions and Answers

1. What is a Kubernetes Service?

A stable network abstraction that exposes a set of matching Pods through a virtual address and, commonly, DNS.

2. What is the difference between ClusterIP and NodePort?

ClusterIP provides internal cluster access. NodePort exposes a Service through a port on cluster nodes, subject to the networking environment.

3. What does a LoadBalancer Service do?

It requests a load-balancing implementation from the underlying environment, when supported, to expose the Service externally.

4. What is the difference between `port` and `targetPort`?

`port` is the Service's exposed port; `targetPort` is the destination port on the selected Pod.

5. How does Kubernetes DNS work for Services?

Cluster DNS resolves Service names to Service addresses. Fully qualified names include the Service, Namespace, and cluster DNS domain.

6. Why should an application avoid using another Pod's IP directly?

Pods can be replaced and receive new IPs. A Service provides a more stable destination.

7. What is Ingress?

A Kubernetes API resource for declaring HTTP and HTTPS routing rules to Services.

8. What is an Ingress controller?

The component that implements Ingress rules. An Ingress resource alone does not provide traffic routing.

9. What is a ConfigMap?

A Kubernetes object for storing non-confidential configuration data.

10. What is a Secret?

A Kubernetes object intended for sensitive configuration, such as credentials. Proper access control and storage protections are still necessary.

11. Is Base64 encryption?

No. Base64 is an encoding format, not encryption.

12. What is a PersistentVolume?

A cluster storage resource representing provisioned storage.

13. What is a PersistentVolumeClaim?

A request for storage with specified requirements.

14. What is a StorageClass?

A resource describing a class of storage and its provisioning behavior.

15. Does a PVC guarantee backups?

No. Persistence and backup/recovery are different requirements.

16. Why might a Service have no endpoints?

Its selector may not match any Pods, or matching Pods may not be eligible endpoints because of their state or readiness.

17. Why does `localhost` fail when connecting to another Pod?

`localhost` refers to the current network namespace, not the other Pod.

18. What is the difference between readiness and liveness probes?

Readiness controls whether a container should receive traffic; liveness helps determine whether a container should be restarted.

19. Why might a ConfigMap update not change the running application?

The application may use environment variables populated at startup, cache configuration, or lack file-reload behavior.

20. How would you debug an API-to-database connection failure?

Verify the Service, selector, endpoints, DNS, network policies, port, process listener, credentials, and application logs in sequence.

19. Hands-On Capstone

Extend the Orders API deployment to include service discovery and external access planning.

Required tasks

- [ ] Create an internal Redis Service.
- [ ] Configure the Orders API using a ConfigMap.
- [ ] Create a development-only Secret using a safe local workflow.
- [ ] Confirm that the API can resolve the Redis Service name.
- [ ] Inspect Service EndpointSlices.
- [ ] Test a real API endpoint using port forwarding.
- [ ] Create a PVC and inspect its binding status.
- [ ] Explain what happens when the Pod is recreated.
- [ ] Design an Ingress rule and identify the controller required to implement it.
- [ ] Diagnose a deliberate Service selector mismatch.
- [ ] Explain why a Secret is not automatically secure merely because it is a Secret.
- [ ] Write a short incident report for a connectivity or storage failure.

Completion standard

You can trace traffic from a client to a Pod, explain how applications find each other, manage configuration separately from application images, and distinguish persistent storage from a complete backup strategy.

Final Mastery Checklist

- [ ] Explain Service networking and stable Pod access.
- [ ] Distinguish ClusterIP, NodePort, and LoadBalancer.
- [ ] Explain Kubernetes Service DNS.
- [ ] Distinguish Ingress resources from Ingress controllers.
- [ ] Configure and troubleshoot ConfigMaps.
- [ ] Use Secrets with least-privilege access.
- [ ] Explain PVs, PVCs, and StorageClasses.
- [ ] Understand why persistence is not backup.
- [ ] Troubleshoot selector mismatches and missing endpoints.
- [ ] Explain readiness, liveness, and startup probes.
- [ ] Diagnose connectivity and configuration incidents.
- [ ] Complete the hands-on capstone.

Kubernetes Handbook Progress

This is **Chunk 2 of 3**.

The remaining planned chapter is:

Chunk 3 — Production Reliability, Security, Autoscaling, Observability, Incident Mastery & Interview Preparation

Save this chapter as:

`content/kubernetes/02-kubernetes-services-networking-ingress-configuration-storage.md`

Do not generate the Kubernetes PDF yet. Save and verify all three planned chapters first, then generate the single complete handbook at:

`docs/kubernetes/BEH_KUBERNETES_COMPLETE_HANDBOOK.pdf`

After all three source chapters and the final PDF are saved and verified, commit with the exact message:

`Kubernetes done`

Do not claim repository changes, PDF generation, or Git commits until they have actually been performed and verified.

Part III: Production Reliability, Security, Autoscaling, Observability & Incident Mastery

Chunk 3 of 3 · Backend Engineering Handbook

Kubernetes — Chunk 3 of 3: Production Reliability, Security, Autoscaling, Observability & Incident Mastery

Handbook: Backend Engineering Tools & Infrastructure

Source file:
`content/kubernetes/03-kubernetes-production-reliability-security-autoscaling-observability.md`

Learning objective: Move from deploying applications on Kubernetes to operating them safely under production conditions. By the end, you should be able to reason about availability, resource pressure, autoscaling, secure workloads, deployment rollbacks, observability, and incident response.

Layer 1 — Fundamentals & Mental Models

1. The production Kubernetes mental model

A Deployment running successfully is not the same as a production-ready service.

A production workload must handle:

- Process crashes and node failures.
- Traffic spikes and changing resource demand.
- Slow dependencies and overloaded databases.
- Bad releases and configuration mistakes.
- Unauthorized access and compromised credentials.
- Partial outages where some replicas work and others fail.
- Limited visibility into what is actually happening.

Think of Kubernetes production engineering as five connected systems:

System	Primary question
Reliability	Can the application continue serving useful traffic?
Resource management	Does it have enough CPU and memory without wasting capacity?
Security	Who can access the workload, and what can it access?

System	Primary question
Observability	Can we detect and diagnose failures?
Delivery	Can we release and reverse changes safely?

Kubernetes provides mechanisms for these goals. It does not automatically guarantee that your application is reliable, secure, or observable.

2. Desired state, reconciliation, and recovery

Suppose a Deployment requests three replicas.

```
spec:
  replicas: 3
```

Kubernetes continuously compares the desired state with the observed state.

If one Pod disappears, the Deployment's controllers work toward restoring the desired replica count.

However, there is an important distinction:

A replacement Pod existing does not mean the service is healthy.

The replacement may fail to start, fail its readiness probe, or be unable to reach PostgreSQL. The Pod count may eventually return to three while the application remains unusable.

Production readiness therefore requires more than replica counts. It requires meaningful health checks, correct traffic routing, suitable resources, safe deployment strategies, and observable behavior.

3. Requests, limits, and resource pressure

Kubernetes resource settings influence scheduling and runtime behavior.

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "1000m"
    memory: "512Mi"
```

CPU

- **Request:** The CPU amount used when Kubernetes evaluates scheduling requirements.
- **Limit:** The CPU usage ceiling enforced by the container runtime and Linux controls.
- CPU limits can cause throttling when the container attempts to use more CPU than allowed.

A throttled application may respond slowly even when its code has not changed.

Memory

- **Request:** Helps Kubernetes decide where the Pod can be scheduled.
- **Limit:** Establishes a memory ceiling.

- Exceeding the memory limit can cause the container to be terminated, commonly showing an `OOMKilled` reason.

Memory pressure and CPU pressure behave differently. CPU throttling often manifests as increased latency; memory exhaustion may terminate the process.

How do you choose resource values?

Start with measurements, not guesses.

1. Measure normal CPU and memory consumption.
2. Exercise realistic request volume.
3. Observe peak usage and garbage collection behavior.
4. Set requests that reflect actual workload needs.
5. Set limits according to the workload's risk and operating requirements.
6. Reassess after load tests and production telemetry.

For Java services, include heap, native memory, thread stacks, and other process overhead when sizing a container. A JVM heap configured too close to the container memory limit can leave insufficient room for non-heap memory.

4. Probes: startup, readiness, and liveness

These probes answer different questions.

Probe	Question	Failure consequence
Startup	Has the application finished starting?	Container may be restarted after the configured startup threshold
Readiness	Should this Pod receive service traffic?	Pod is removed from ready endpoints while unready
Liveness	Is the process sufficiently healthy to continue running?	Container may be restarted

Example:

```
startupProbe:
  httpGet:
    path: /actuator/health
    port: 8080
  periodSeconds: 5
  failureThreshold: 30

readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  periodSeconds: 5
  failureThreshold: 3

livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  periodSeconds: 10
  failureThreshold: 3
```

These endpoints must actually exist and have suitable semantics in the application. For Spring Boot, configure Actuator health probes appropriately for the deployed version and dependencies.

Critical distinction: Do not make liveness depend blindly on every external dependency. If PostgreSQL becomes temporarily unavailable and the liveness endpoint fails for that reason, Kubernetes may repeatedly restart otherwise functional application instances. That can amplify the outage.

Readiness is often the right mechanism to stop sending traffic to an instance that cannot serve requests. Whether a dependency failure should make an application unready depends on the API's behavior and its ability to serve degraded functionality.

5. Horizontal Pod Autoscaling

The Horizontal Pod Autoscaler (HPA) adjusts the number of Pod replicas based on observed metrics.

```
Incoming traffic increases
          |
          v
Resource usage or custom metric increases
          |
          v
HPA evaluates its target
          |
          v
Desired replica count increases
          |
          v
Deployment creates more Pods
          |
          v
Scheduler places Pods on available nodes
```

Autoscaling is a control loop, not an instant reaction. Metrics must be available, the HPA must observe the threshold, Pods must start, and the cluster must have sufficient capacity.

A basic CPU-based HPA:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend-api
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend-api
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

CPU utilization is measured relative to CPU requests. Missing or unsuitable requests can make CPU-utilization-based scaling behave unexpectedly.

HPA also requires a working metrics pipeline, commonly provided by Metrics Server for resource metrics. Metrics Server is not a replacement for a full monitoring system such as Prometheus.

When CPU is not the right scaling signal

A backend can be overloaded without high CPU usage.

Examples:

- Requests wait on a database connection pool.
- The application spends time waiting on network calls.
- Kafka consumer lag grows while CPU remains moderate.
- The number of requests in flight increases.
- Queue depth grows faster than workers can process it.

Custom or external metrics may be more appropriate in these cases, provided they are exposed and integrated correctly.

Autoscaling cannot solve every bottleneck. Ten additional API Pods may worsen a database connection storm if every Pod opens too many connections.

6. PodDisruptionBudgets and availability

A PodDisruptionBudget (PDB) limits how many matching Pods can be voluntarily disrupted at once.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: backend-api-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: backend-api
```

This example aims to keep at least two matching Pods available during supported voluntary disruptions, assuming enough healthy replicas exist.

PDBs can help with operations such as node maintenance and certain drain workflows. They do not guarantee availability during every failure.

A PDB does not prevent:

- A node crashing unexpectedly.
- An application bug affecting every replica.
- A bad configuration breaking all Pods.
- A database outage.
- A failed readiness configuration.

For availability, combine multiple replicas, correct readiness checks, appropriate placement, safe deployments, and resilient dependencies.

7. Rolling updates and rollbacks

A Deployment can replace old Pods gradually.

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 0
    maxSurge: 1
```

- `maxUnavailable: 0` aims to avoid reducing the number of available replicas during the rollout.
- `maxSurge: 1` permits one additional Pod above the desired replica count during the rollout.

This configuration requires enough cluster capacity and compatible application behavior. It cannot guarantee zero downtime if the application, database migration, or dependencies are not compatible with the transition.

Inspect rollout status:

```
kubectl rollout status deployment/backend-api
kubectl rollout history deployment/backend-api
kubectl describe deployment backend-api
```

Undo a rollout:

```
kubectl rollout undo deployment/backend-api
```

A rollback changes the Deployment's Pod template to an earlier revision. It does **not** automatically reverse database migrations, external side effects, or changes to persistent data.

Expand-and-contract database migrations

For backward-compatible releases:

1. Add a new schema element without immediately removing the old one.
2. Deploy code that can work with both schemas.
3. Migrate or backfill data.
4. Verify that old code paths are no longer needed.
5. Remove obsolete schema elements in a later release.

This approach makes it safer to roll application code forward and backward during a deployment window.

Layer 2 — Practical Engineering & Workflows

8. Production-minded Deployment template

The following is an illustrative Deployment template for a Spring Boot API. Replace the image reference, endpoints, and resource values with settings verified for your application.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-api
spec:
  replicas: 3
  revisionHistoryLimit: 5
  selector:
    matchLabels:
      app: backend-api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  template:
    metadata:
      labels:
        app: backend-api
    spec:
      automountServiceAccountToken: false
      terminationGracePeriodSeconds: 30
      securityContext:
        runAsNonRoot: true
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: api
          image: your-registry/backend-api:1.0.0
          ports:
            - name: http
              containerPort: 8080
          resources:
            requests:
              cpu: "250m"
```

```
            memory: "256Mi"
          limits:
            cpu: "1000m"
            memory: "512Mi"
        startupProbe:
          httpGet:
            path: /actuator/health
            port: http
          periodSeconds: 5
          failureThreshold: 30
        readinessProbe:
          httpGet:
            path: /actuator/health/readiness
            port: http
          periodSeconds: 5
          failureThreshold: 3
        livenessProbe:
          httpGet:
            path: /actuator/health/liveness
            port: http
          periodSeconds: 10
          failureThreshold: 3
        securityContext:
          allowPrivilegeEscalation: false
          readOnlyRootFilesystem: true
          capabilities:
            drop:
              - ALL
```

Important caveats

- The image must be accessible from the cluster, and the image version should be immutable or otherwise reliably traceable.
- The application must support running as a non-root user.
- A read-only root filesystem may require writable temporary or application-specific volumes.
- Spring Boot must expose the configured health endpoints.
- Requests, limits, replicas, and probe thresholds require workload-specific validation.
- `automountServiceAccountToken: false` is appropriate when the workload does not need to call the Kubernetes API using its service account.
- This manifest alone does not create a Service, HPA, PDB, NetworkPolicy, Secret, or Ingress.

Apply and verify:

```
kubectl apply -f deployment.yaml
kubectl rollout status deployment/backend-api
kubectl get deployment backend-api
kubectl get pods -l app=backend-api -o wide
kubectl describe deployment backend-api
```

9. Security: identity, permissions, and isolation

Kubernetes security should follow least privilege.

Service accounts

A ServiceAccount gives a workload an identity when it needs to interact with the Kubernetes API or other identity-integrated systems.

Do not grant broad permissions simply because an application runs inside the cluster.

If the application does not need Kubernetes API access, consider disabling automatic service-account token mounting. If access is required, create a narrowly scoped Role or ClusterRole and bind it only to the necessary identity.

RBAC mental model

- **Role:** Permissions within a namespace.
- **ClusterRole:** A cluster-scoped permission definition, or a reusable permission definition that can be bound at namespace scope.
- **RoleBinding:** Grants permissions in a namespace.
- **ClusterRoleBinding:** Grants permissions across the cluster.

Example: allow a dedicated service account to read Pods in one namespace.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: diagnostics-reader
  namespace: default
automountServiceAccountToken: true
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: default
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: diagnostics-reader-binding
  namespace: default
subjects:
  - kind: ServiceAccount
    name: diagnostics-reader
    namespace: default
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: pod-reader
```

Validate permissions before deploying:

```
kubectl auth can-i get pods \
  --as=system:serviceaccount:default:diagnostics-reader \
  -n default
```

Run this check with an identity that is authorized to impersonate the service account. The check does not itself grant permissions.

NetworkPolicy

NetworkPolicy can restrict Pod ingress and egress when the cluster's networking implementation supports enforcement.

A default-deny policy can improve isolation, but it can also break DNS, API access, database connections, or monitoring if necessary traffic is not explicitly allowed.

A safe rollout sequence is:

1. Inventory legitimate network flows.
2. Identify application, DNS, database, and monitoring dependencies.
3. Apply policies in a test namespace.
4. Verify expected traffic and denied traffic.
5. Roll out gradually while monitoring errors.

Do not apply an untested cluster-wide deny-all policy to production.

Secrets

Kubernetes Secret data is Base64-encoded in manifests; Base64 is not encryption.

Production secret management should consider:

- Encryption at rest configured for the cluster.
- RBAC restrictions on reading and modifying Secrets.
- External secret managers where appropriate.
- Secret rotation and revocation.
- Avoiding secrets in Git, container images, logs, and shell history.

10. Graceful shutdown and termination

When a Pod is terminated, Kubernetes can remove it from service endpoints and send termination signals to the container. The precise sequence and timing depend on the workload and platform.

A well-behaved backend should:

1. Stop accepting new work when shutdown begins.
2. Become unready so new requests are no longer routed to it.
3. Allow in-flight requests to finish within a bounded period.
4. Close database and messaging connections cleanly.
5. Exit before the termination grace period expires.

For Java services, verify Spring Boot's graceful-shutdown configuration and ensure the application server's shutdown behavior matches the service's request duration.

For Kafka consumers, shutdown may also require completing or safely abandoning in-flight processing and committing offsets only when the processing semantics permit it.

A Kubernetes grace period cannot make an application graceful if the application ignores termination signals or blocks indefinitely.

11. Observability: metrics, logs, and traces

These three signals answer different questions.

Signal	Best for
Metrics	Trends, alerting, saturation, error rates, latency distributions
Logs	Detailed events and contextual error information
Traces	Following a request across services and dependencies

Useful Kubernetes-level signals include:

- Pod restart count.
- Container CPU throttling.
- Memory working set and OOM events.
- Desired versus available replicas.
- Pending Pods and scheduling failures.
- HPA current versus desired replicas.
- Node CPU, memory, and disk pressure.

Application-level signals include:

- Request rate.
- Error rate.
- Request latency percentiles.
- Database pool saturation.
- Dependency timeout rates.
- Queue depth or Kafka consumer lag.

A healthy Pod is not proof of a healthy API. Combine infrastructure and application telemetry.

Practical investigation commands

```
kubectl get pods -A
kubectl get events -A --sort-by=.lastTimestamp
kubectl top nodes
kubectl top pods -n default
kubectl logs deployment/backend-api --since=15m
kubectl logs <pod-name> --previous
kubectl describe pod <pod-name>
```

`kubectl top` requires a functioning resource metrics pipeline. `--previous` is especially useful after a container has restarted because it retrieves logs from the previous container instance when available.

12. Autoscaling and availability configuration

Create an HPA:

```
kubectl autoscale deployment backend-api \
  --cpu-percent=70 \
  --min=2 \
  --max=10
```

Inspect it:

```
kubectl get hpa
kubectl describe hpa backend-api
kubectl top pods
```

The command creates an autoscaler targeting CPU utilization. It will not behave as intended if the metrics pipeline or resource requests are unsuitable.

Check the Deployment's available replicas:

```
kubectl get deployment backend-api \
  -o custom-columns=DESIRED:.spec.replicas,READY:.status.readyReplicas,AVAILABLE:.status
.availableReplicas
```

Check disruption budgets:

```
kubectl get pdb
kubectl describe pdb backend-api-pdb
```

Check rollout health:

```
kubectl rollout status deployment/backend-api
kubectl rollout history deployment/backend-api
```

13. A disciplined release workflow

Use this sequence for a backend deployment.

Before the release

- Verify the build and test suite.
- Use an identifiable, immutable image reference.
- Check resource requests and limits.
- Verify health probes.
- Confirm database migration compatibility.
- Review Secret and RBAC changes.
- Confirm enough cluster capacity for rollout surge.
- Check alerting and dashboards.

During the release

- Observe rollout progress.
- Watch available replicas and Pod restarts.
- Check request error rate and latency.

- Verify dependency health.
- Stop or roll back if the release crosses agreed failure thresholds.

After the release

- Verify API behavior using a smoke test.
- Check error and latency trends against baseline.
- Confirm consumers and background jobs are progressing.
- Record the deployed version and any deviations.
- Keep the incident timeline if anything went wrong.

A rollout reaching `successfully rolled out` is a control-plane status, not proof that business functionality is correct.

Layer 3 — Production Engineering & Interview Mastery

14. Incident simulation: a release causes HTTP 5xx errors

Scenario: A new version deploys. The Pods are running, but the API's error rate increases.

Step 1: Establish impact

Check the customer-facing symptoms, affected endpoints, and start time. Compare the start time with the release timeline.

Step 2: Inspect the rollout

```
kubectl rollout status deployment/backend-api
kubectl rollout history deployment/backend-api
kubectl get replicasets
kubectl get pods -l app=backend-api
```

Step 3: Inspect new Pods

```
kubectl describe pod <new-pod-name>
kubectl logs <new-pod-name> --since=10m
```

Look for startup failures, connection errors, missing configuration, and dependency timeouts.

Step 4: Separate application failure from routing failure

Check whether Pods are ready and whether the Service has healthy endpoints.

```
kubectl get service
kubectl get endpointslice \
  -l kubernetes.io/service-name=backend-api
```

If no endpoints are ready, investigate readiness probes and Pod state. If endpoints exist, continue into application logs, dependencies, and request traces.

Step 5: Mitigate

If the release is the likely cause and rollback is safe:

```
kubectl rollout undo deployment/backend-api
kubectl rollout status deployment/backend-api
```

Then verify the error rate and user-facing behavior.

Step 6: Learn

Determine whether the failure came from code, configuration, migration incompatibility, resource settings, or an overlooked dependency. Add a test, release guard, or alert that would catch the failure earlier.

Interview signal: You establish impact, inspect evidence, mitigate, and verify recovery instead of randomly restarting Pods.

15. Incident simulation: Pods are repeatedly OOMKilled

Scenario: A Spring Boot service restarts under load.

Inspect:

```
kubectl describe pod <pod-name>
kubectl get pod <pod-name> -o yaml
kubectl top pod <pod-name>
kubectl logs <pod-name> --previous
```

Investigate:

- Container memory limit.
- JVM heap and non-heap memory.
- Memory growth over time.
- Traffic or workload changes.
- Large payloads and unbounded caches.
- Thread stacks, direct buffers, and native allocations.
- Whether the Pod is on a memory-pressured node.

Possible responses:

- Correct an unrealistically low memory limit when evidence supports it.
- Reduce excessive memory use in the application.
- Tune the JVM with container-aware memory sizing.
- Bound caches and queues.
- Add load testing and memory monitoring.

Simply increasing the memory limit may postpone the incident without fixing a leak.

16. Incident simulation: HPA does not scale

Scenario: Traffic rises, but the replica count remains unchanged.

Check:

```
kubectl get hpa
kubectl describe hpa backend-api
kubectl top pods
kubectl get deployment backend-api
kubectl get events --sort-by=.lastTimestamp
```

Look for:

- Missing metrics.
- Missing CPU requests.
- Current utilization below the configured target.
- An HPA at its maximum replica count.
- Unschedulable Pods due to insufficient node capacity.
- Startup time too long for the traffic spike.
- The real bottleneck being a database, queue, or external dependency.

Distinguish **scaling not triggered** from **scaling triggered but Pods cannot be scheduled**. They require different fixes.

17. Incident simulation: rolling deployment stalls

Scenario: New Pods never become available.

Inspect:

```
kubectl rollout status deployment/backend-api
kubectl describe deployment backend-api
kubectl get pods -l app=backend-api
kubectl describe pod <new-pod-name>
kubectl get events --sort-by=.lastTimestamp
```

Common causes:

- Image cannot be pulled.
- Startup or readiness probe is incorrect.
- Resource requests cannot fit on available nodes.
- Application configuration is missing.
- Dependency initialization blocks startup.
- The new version is incompatible with the existing database schema.
- Security settings prevent the process from starting.

Do not assume the correct fix is to increase rollout timeouts. Identify the actual blocker first.

18. Reliability engineering: error budgets and SLOs

An **SLO** is a target for a service-level indicator over a defined time window.

For example:

- Availability SLI: successful eligible requests divided by total eligible requests.
- Latency SLI: fraction of eligible requests completed below a latency threshold.
- SLO: 99.9% of eligible requests succeed over a rolling 30-day window.

An error budget is the permitted unreliability implied by the SLO.

For a 99.9% availability target, the theoretical error budget is 0.1% of eligible requests over the selected window. That is not automatically a fixed number of minutes: the conversion depends on how availability is measured and the window used.

SLOs help teams make release and reliability tradeoffs. They do not replace diagnosing incidents or selecting useful alerts.

Alert on symptoms, investigate causes

A high-value alert often identifies user-visible impact, such as sustained elevated error rates or latency. CPU usage, restart count, and disk pressure are useful diagnostic signals, but not every elevated resource metric represents a customer-impacting incident.

Avoid alerting on every brief fluctuation. Define the symptom, threshold, duration, ownership, and required response.

19. Production readiness checklist

Use this before calling a workload production-ready.

Availability and delivery

- [ ] Multiple replicas where required.
- [ ] Readiness, liveness, and startup probes have appropriate semantics.
- [ ] Rolling updates have suitable surge and unavailability settings.
- [ ] Rollback has been tested.
- [ ] Database migrations support the intended release sequence.
- [ ] Pod placement and cluster capacity match availability goals.
- [ ] PDB settings reflect actual disruption requirements.

Resources and scaling

- [ ] CPU and memory requests are evidence-based.
- [ ] Limits are appropriate for the workload.
- [ ] JVM and container memory are sized together.
- [ ] HPA metrics are available and validated.
- [ ] Scaling behavior has been load-tested.
- [ ] Database and other dependency capacity are considered.

Security

- [ ] Containers run as non-root when feasible.
- [ ] Privilege escalation is disabled unless explicitly needed.
- [ ] Linux capabilities are minimized.
- [ ] Service account permissions follow least privilege.
- [ ] Unneeded service-account token mounting is disabled.
- [ ] Secrets are not committed to source control.
- [ ] Network policies are tested and enforced by the cluster's network implementation.
- [ ] Image provenance, patching, and vulnerability scanning have appropriate controls.

Observability and incident response

- [ ] Request rate, error rate, and latency are visible.
- [ ] Restarts, OOM events, CPU throttling, and pending Pods are observable.
- [ ] Logs provide enough context to diagnose failures.
- [ ] Distributed tracing is available where useful.
- [ ] Alerts have clear ownership and actionable runbooks.
- [ ] Rollback and recovery procedures are documented.
- [ ] A failure simulation has been completed.

---

20. SDE-1 interview questions and answers

Fundamentals

1. Does Kubernetes automatically make an application highly available?

No. Kubernetes provides mechanisms such as replicas, rescheduling, probes, and rolling updates. Availability still depends on application design, placement, dependencies, capacity, and configuration.

2. What is the difference between a readiness probe and a liveness probe?

Readiness determines whether a Pod should receive traffic. Liveness determines whether Kubernetes should restart a container because it is considered unhealthy.

3. Why are startup probes useful?

They allow slow-starting applications time to initialize before liveness checks begin enforcing failure thresholds.

4. What is the difference between a CPU request and a CPU limit?

A request informs scheduling and contributes to utilization calculations. A limit caps CPU consumption and may cause throttling.

5. What happens when a container exceeds its memory limit?

It may be terminated by the kernel, often resulting in an `OOMKilled` status. Inspect Pod details and previous logs to confirm the failure.

Autoscaling and availability

6. How does an HPA work?

It periodically compares observed metrics with configured targets and adjusts the desired replica count within configured bounds.

7. Why might an HPA fail to scale a Deployment?

Metrics may be unavailable, requests may be missing, utilization may be below the target, the maximum replica count may have been reached, or new Pods may be unschedulable.

8. Does a PodDisruptionBudget prevent a node failure?

No. It constrains supported voluntary disruptions, not arbitrary involuntary failures.

9. How do you reduce downtime during rolling updates?

Use suitable readiness probes, enough replicas and capacity, compatible application and database changes, and an appropriate rollout strategy. Validate real behavior during the release.

10. Why is rollback not always sufficient?

The previous application version may be incompatible with a database migration, persistent data change, or irreversible external side effect.

Security and networking

11. What is RBAC?

Role-Based Access Control governs which identities can perform which operations on Kubernetes resources.

12. Why should applications avoid broad cluster-admin access?

Excessive privileges increase the impact of a compromised workload or application vulnerability.

13. Is Base64-encoded Kubernetes Secret data encrypted?

No. Base64 is encoding, not encryption. Access control and suitable encryption-at-rest configuration are important.

14. What is the purpose of a NetworkPolicy?

It specifies allowed Pod network traffic. Enforcement depends on the cluster networking implementation.

15. Why might a container run as non-root?

It reduces the privileges available to a compromised process and supports defense in depth.

Operations and debugging

16. A Pod is Running but not Ready. What do you inspect?

Inspect the readiness probe, Pod events, container logs, application health endpoint, configuration, and dependencies required for serving requests.

17. What does CrashLoopBackOff mean?

The container repeatedly exits and Kubernetes backs off before attempting another restart. It is a symptom; inspect exit status, events, and logs to find the cause.

18. How do you inspect logs from a crashed container?

Use `kubectl logs <pod-name> --previous` when the previous container's logs are available.

19. How do metrics, logs, and traces complement one another?

Metrics identify patterns and symptoms, logs provide detailed event context, and traces show the path and timing of requests across components.

20. What is your first response to a production deployment failure?

Establish user impact, correlate it with the release, inspect rollout status and evidence, mitigate safely, verify recovery, and then conduct a root-cause analysis.

---

21. Capstone: operate a Spring Boot API under failure

Objective

Deploy and operate a backend API locally on a Kubernetes development cluster, such as Kind or Minikube. No paid cloud account is required.

Stage A — Prepare

1. Build and test the Spring Boot application.
2. Build a container image and make it accessible to the local cluster.
3. Deploy at least two replicas.
4. Create a ClusterIP Service.
5. Configure startup, readiness, and liveness probes.
6. Set CPU and memory requests.

Stage B — Introduce operational controls

1. Configure a rolling update strategy.
2. Create a PDB appropriate to the replica count.
3. Configure an HPA if the cluster has the required metrics support.
4. Run a least-privilege RBAC check.
5. Test shutdown and restart behavior.
6. Add metrics and structured application logs.

Stage C — Run failure drills

Perform these drills only in your local development environment.

- Deploy an image tag that does not exist, then diagnose `ImagePullBackOff`.
- Configure a deliberately incorrect readiness path and observe the effect on endpoints.
- Set a low memory limit in a test environment and observe the resulting termination behavior under a controlled workload.
- Deploy a deliberately broken application version and practice a rollback.
- Remove a required environment variable and diagnose startup failure.
- Generate test traffic and observe whether HPA metrics and scaling behave as expected.

Do not use destructive drills on a production cluster.

Stage D — Write a short incident report

For each drill, record:

- Impact and symptoms.
- Detection method.
- Commands and evidence collected.
- Root cause.
- Mitigation and verification.
- Preventive change.
- What you would explain in an interview.

Stage E — Mastery test

You have completed this chapter when you can explain and demonstrate:

- Why a Running Pod can still be unhealthy.
- How readiness differs from liveness.
- Why CPU throttling and memory exhaustion have different symptoms.
- What an HPA needs in order to scale.
- What a PDB does and does not guarantee.
- How to roll back a bad release safely.
- Why Kubernetes Secrets need additional security controls.
- How to narrow down a failed rollout using evidence.
- How metrics, logs, and traces work together.
- How to explain the incident clearly to an interviewer.

Kubernetes completion protocol

This is the final planned Kubernetes content chunk.

1. Save this chapter as `content/kubernetes/03-kubernetes-production-reliability-security-autoscaling-observability.md`.
2. Verify that all three planned Kubernetes Markdown chapters exist and contain their complete content.
3. Build **one complete Kubernetes handbook PDF** from all three chapters in curriculum order.
4. Save the PDF as `docs/kubernetes/BEH_KUBERNETES_COMPLETE_HANDBOOK.pdf`.
5. Verify the PDF's chapter order, headings, code blocks, tables, and page rendering.
6. Update the README only with progress that has actually been verified.
7. Commit the completed Kubernetes source chapters and verified PDF with the exact commit message:

```
Kubernetes done
```

Do not generate a PDF for this chunk alone. The three source chapters and the single final Kubernetes PDF form the complete handbook deliverable.

After this is saved and verified, the next planned tool in the curriculum is Terraform, with three chunks. The overall project remains incomplete until all 13 individual handbooks and the final master handbook are complete.